



ELABORAÇÃO DE ALGORITMOS

01 - Crie uma função somaPares que receba um inteiro N e retorne a soma dos N primeiros números pares positivos ($2 + 4 + 6 + \dots$ até totalizar N termos). Na main, leia N e exiba o resultado retornado pela função.

Repetição: for

Por quê? o número de iterações é conhecido previamente (N termos), e o for é a estrutura mais natural para contagem fixa com contador, início e passo bem definidos.

Condicional: if

Por quê? será usada apenas para validar se N é positivo (validação simples de uma condição).

02 - Implemente a função classificarIMC que recebe o peso (kg) e a altura (m) de uma pessoa, calcula o $IMC = peso / (altura * altura)$ e imprime a faixa correspondente: “Abaixo do peso” (< 18.5), “Normal” (18.5 a 24.9), “Sobrepeso” (25 a 29.9), “Obesidade” (≥ 30). Na main, leia peso e altura e chame a função.

Repetição: Não necessita

Por quê? Esta questão não exige repetição: o cálculo é direto, baseado em entrada única.

Condicional: if/else encadeados (else if)

Por quê? As faixas do IMC são intervalos contínuos com limites variados (não são valores discretos), por isso o switch-case não se aplica bem.

03 - Crie a função converterComprimento que recebe um valor (*float*) em metros e um código inteiro indicando a unidade de destino: 1 = quilômetros, 2 = centímetros, 3 = milímetros, 4 = polegadas ($1\text{ m} = 39.3701\text{ pol}$). A função deve imprimir o valor convertido. Na main, leia o valor e o código e chame a função.

Repetição: Não necessita

Por quê? Não há iteração: cada execução faz uma única conversão.

Condicional: switch-case

Por quê? A escolha se dá entre um conjunto fechado e pequeno de valores inteiros discretos (1, 2, 3, 4), situação ideal para switch-case, que torna o código mais limpo do que uma cadeia de if/else.

04 - Implemente a função `lerSenhaValida` que solicita repetidamente ao usuário uma senha numérica de 4 dígitos (1000 a 9999) e só retorna quando o valor digitado estiver dentro dessa faixa. A função deve retornar a senha válida. Na main, chame a função e mostre a senha aceita.

Repetição: do-while

Por quê? É preciso pedir a senha pelo menos uma vez antes de testar a condição. O `do-while` executa antes de avaliar.

Condicional: if

Por quê? Uma única condição binária (senha está ou não no intervalo) controla o laço.

05 - Crie a função `contarDigitos` que recebe um número inteiro positivo e retorna a quantidade de dígitos dele (ex.: 4587 -> 4 dígitos). Não use conversão para *string*. Na main, leia um número e exiba a contagem.

Repetição: while

Por quê? Não sabemos previamente quantas iterações ocorrerão. O `while` é ideal quando o número de repetições depende de uma condição lógica, não de um contador conhecido.

Condicional: if

Por quê? Será usada apenas para tratar o caso especial em que o número informado é 0 (que tem 1 dígito).

06 - Implemente a função `imprimirDiaSemana` que recebe um inteiro de 1 a 7 e imprime o nome do dia correspondente: 1 = “Domingo”, 2 = “Segunda-feira”, ..., 7 = “Sábado”. Se o valor não estiver no intervalo, deve exibir “Dia inválido”. Na main, leia um inteiro e chame a função.

Repetição: Não necessita

Por quê? A função é executada uma única vez por chamada, sem iteração.

Condicional: switch-case

Por quê? Trata-se de mapear valores inteiros discretos (1 a 7) para mensagens, mais legível e direto que uma sequência de `if/else if`.

07 - Crie a função `inverterNumero` que recebe um número inteiro positivo e retorna seu inverso (ex.: 1234 -> 4321). Use operações matemáticas (módulo e divisão inteira). Na main, leia um número e mostre seu inverso pelo módulo.

Repetição: while

Por quê? O laço deve continuar até que o número original seja reduzido a 0, não de um contador conhecido.

Condicional: if

Por quê? Usada apenas para validar se o número informado é positivo.

08 - Implemente a função `ehBissexto` que recebe um ano (int) e retorna 1 se for bissexto e 0 caso contrário. Regra: divisível por 4 e (não divisível por 100 ou divisível por 400). Na main, leia o ano e informe ao usuário se é bissexto.

Repetição: Não necessita

Por quê? Verificação direta de uma condição lógica composta, sem necessidade de iteração.

Condicional: if

Por quê? A regra é uma expressão booleana composta (com `&&` e `||`). O `if` é a estrutura natural para avaliar essa expressão; `switch` só serve para igualdade com valores fixos.

09 - Crie a função `tabelaPotencias` que recebe dois inteiros, início e fim, e imprime para cada número desse intervalo (inclusive) seu quadrado e seu cubo, em formato de tabela. Na main, leia os dois valores e chame a função.

Repetição: for

Por quê? O intervalo de iteração é conhecido (de início até fim)

Condicional: if

Por quê? Uma única verificação para garantir que `inicio <= fim` antes de iterar.

10 – Implemente a função `analisarSequencia` que lê números reais (float) digitados pelo usuário e os processa até que o usuário digite zero (zero não entra na conta). A função deve imprimir o maior, o menor e a média dos valores lidos. Na main, apenas chame a função.

Repetição: do-while

Por quê? É necessário ler ao menos uma vez antes de verificar se o usuário digitou zero. Como a condição de parada depende do dado lido, o `do-while` é mais natural que `while` ou `for`.

Condicional: if

Por quê? Diversas comparações em sequência (atualizar maior, menor, somar)

11 – Crie a função `executarOperacao` que recebe dois números reais e um caractere de operação ('+', '-', '*', '/') e imprime o resultado. Trate divisão por zero e operação inválida. Na main, mostre um menu de operações, leia os dois números e o caractere, e chame a função.

Repetição: Não necessita

Por quê? a função realiza uma única operação por chamada; não há laço dentro dela.

(Caso queira permitir várias operações, a repetição pode ser feita na main, mas não é o foco da questão.)

Condicional: switch-case

Por quê? O caractere de operação tem um conjunto pequeno e fixo de valores válidos ('+', '-', '*', '/'). Switch-case com case '+': ... é a forma mais limpa.

12 – Implemente a função mdc que recebe dois inteiros positivos a e b e retorna o Máximo Divisor Comum usando o algoritmo de Euclides (substituições sucessivas: enquanto b != 0, faça resto = a % b; a = b; b = resto). Na main, leia os dois valores e exiba o MDC.

Repetição: while

Por quê? A condição de parada (b == 0) é testada no início e o número de iterações é desconhecido, depende dos valores.

Condicional: if

Por quê? Usada para validar a entrada (a e b precisam ser positivos).

13 - Crie a função imprimirFibonacci que recebe um inteiro N (N >= 1) e imprime os N primeiros termos da sequência de Fibonacci (0, 1, 1, 2, 3, 5, 8, 13, ...). Na main, leia N e chame a função.

Repetição: for

Por quê? O número exato de termos a imprimir é conhecido (N).

Condicional: if

Por quê? Necessária para os casos base (N == 1 imprime apenas 0; N == 2 imprime 0 e 1) antes de entrar no laço.

14 – Implemente a função classificarTriangulo que recebe três valores reais a, b, c representando lados de um triângulo. A função deve: (1) verificar se os valores formam um triângulo válido (cada lado < soma dos outros dois); (2) se não formar, imprimir "Não é triângulo"; (3) se formar, classificar quanto aos lados (Equilátero / Isósceles / Escaleno) e quanto aos ângulos usando o recíproco do teorema de Pitágoras (Retângulo se $a^2 = b^2 + c^2$; Obtusângulo se $a^2 > b^2 + c^2$; Acutângulo se $a^2 < b^2 + c^2$, considerando a como o maior lado). Na main, leia os três lados e chame a função.

Repetição: Não há necessidade

Por quê? Tarefa de classificação sobre entrada única.

Condicional: if/else (encadeados)

Por quê? As classificações envolvem comparações entre variáveis (a == b == c, comparações com soma de quadrados etc.).

15 - Crie a função `decimalParaBinario` que recebe um inteiro positivo na base decimal e imprime sua representação em binário usando o método das divisões sucessivas por 2. Dica: armazene os restos em um vetor e imprima na ordem inversa. Na main, leia o número e chame a função.

Repetição: while (para gerar os bits) + for (para imprimir invertido)

Por quê? while: O número de divisões depende do valor de entrada (não é conhecido previamente). For: depois de armazenados, os *bits* precisam ser percorridos do último para o primeiro, com índice controlado.

Condicional: if

Por quê? Necessária para tratar o caso especial em que a entrada é 0 (binário também é 0) antes de iniciar o laço de divisões.

